

GPT-Codex-Guide

《从 ChatGPT 到 Codex——零基础 AI 开发完全指南》

一本面向零基础用户、持续维护的中文开源学习指南，帮助读者理解并连接 ChatGPT、Codex、Git、GitHub、VS Code、Windows 与 macOS。

当前阶段：项目框架与维护规范已经建立。各章保留大纲和后续编写接口，正文将按路线图逐步完善。

项目简介

AI 工具越来越容易使用，但零基础读者仍常被软件、账号、仓库、命令和同步等概念卡住。本项目把分散的知识整理成一条可操作的学习路径：先理解工具为什么存在，再按步骤安装、使用和连接它们。

本书坚持：

- 不默认读者有编程基础。
- 专业术语第一次出现时必须解释。
- 每一步说明做什么、为什么这样做、正确结果是什么以及出错时先检查什么。

本书适合哪些人

- 零基础学习者**：从未写过代码，也不了解 Git 或命令行。
- 办公人员**：希望用 AI 处理文档或自动化重复工作。
- 管理人员**：希望理解 AI 开发流程，更清楚地提出和验收需求。
- 创业者**：希望用较低成本把想法整理成可执行项目。
- AI 学习者**：会用对话工具，希望继续学习 Codex 与项目协作。
- 跨平台用户**：需要在 Windows 与 macOS 间继续同一项目。

整套工具关系图

流程图

```
flowchart TB
    U["用户<br/>提出目标并检查结果"]
    C["ChatGPT（策划）<br/>解释、讨论、整理需求"]
    X["Codex（执行）<br/>读取项目、修改文件、运行检查"]
    G["Git<br/>记录文件变化"]
    H["GitHub<br/>在线保存、分享与协作"]
    O["Windows 与 macOS<br/>运行软件的操作系统"]
    V["VS Code<br/>查看、编辑和管理项目"]
    U --> C --> X --> G --> H --> O --> V
```

这张图表示入门学习顺序，不代表严格的技术上下级关系。例如 VS Code 运行在操作系统上，Git 可以在 VS Code 内或终端中使用，GitHub 保存 Git 仓库的在线副本。

学习路线

章节	学什么	入口
第 1 章：整体认识	认识七种工具及其关系，区分本地与在线	[开始学习](docs/01-整体认识.md)
第 2 章：为什么要学习 ChatGPT 与 Codex	理解学习价值、工具作用与基本协作方法	[开始学习](docs/02-为什么安装这些软件.md)
第 3 章：ChatGPT 是什么	认识 ChatGPT 的能力、边界与正确使用方法	[开始学习](docs/03-Windows安装.md)
第 4 章：Codex 是什么	理解 Codex 的能力、边界与协作模式	[开始学习](docs/04-Mac安装.md)
第 5 章：GitHub 是什么	理解 GitHub、Git 的区别与在线协作作用	[开始学习](docs/05-GitHub使用.md)
第 6 章：Git 使用	理解版本管理，完成提交、拉取和推送	[开始学习](docs/06-Git使用.md)
第 7 章：VS Code 使用	打开项目、编辑 Markdown、使用终端和 Git	[开始学习](docs/07-VSCode使用.md)
第 8 章：Codex 使用	描述任务、限定边界、检查修改并验收	[开始学习](docs/08-Codex使用.md)
第 9 章：ChatGPT 与 Codex 分工	根据任务选择策划或执行工具	[开始学习](docs/09-ChatGPT与Codex分工.md)
第 10 章：Windows 与 Mac 互联互通	在不同电脑间安全同步项目	[开始学习](docs/10-Windows与Mac互联互通.md)
第 11 章：开发固定流程	建立需求、检查、提交、推送的固定循环	[开始学习](docs/11-开发固定流程.md)
第 12 章：常见问题	保存错误信息并按顺序排查故障	[开始学习](docs/12-常见问题.md)
第 13 章：每日操作清单	用开工和收工清单降低遗漏	[开始学习](docs/13-每日操作清单.md)

建议先读第 1、2 章；第 3、4 章选择与你电脑对应的一章；之后按顺序继续。

项目目录说明

路径	作用	为什么这样安排
docs/	存放按学习顺序编号的中文章节	方便顺序学习和单章维护
images/	存放截图和示意图	与正文分开，便于复用和更新
examples/	存放安全练习与示例	让零基础读者放心动手
templates/	存放清单、提示词和模板	减少重复操作，形成固定流程
README.md	项目首页与全书导航	让第一次到访的人快速找到入口
ROADMAP.md	版本路线图	公开说明接下来建设什么
CHANGELOG.md	版本变化记录	说明每个版本新增或调整的内容
CONTRIBUTING.md	贡献规范	统一文件、图片和写作标准
CODE_OF_CONDUCT.md	社区行为准则	维护友好、尊重和开放的环境

版本规划

版本	重点
--- ---	
V1.0	项目框架、首页、章节大纲与维护规范
V2.0	Windows
V3.0	macOS
V4.0	Git
V5.0	GitHub
V6.0	VS Code
V7.0	Codex
V8.0	MCP
V9.0	AI Agent
V10.0	企业开发

详细计划参见 [ROADMAP](#)，版本变化参见 [CHANGELOG](#)。

教材标准与辅助导航

- [分周学习路线](#)：从第一周到完成全书的学习安排。
- [中文术语表](#)：用零基础语言解释全书常用术语。
- [图片与写作风格指南](#)：统一截图、Mermaid、代码块和 Markdown。
- [唯一章节模板](#)：所有新章节必须从此模板开始。

如何参与

欢迎修正错别字、改善解释、补充跨平台步骤或贡献安全示例。提交前请阅读：

- [贡献指南](#)
- [社区行为准则](#)

许可证

本项目采用 [MIT License](#)。

第一章

第1章 整体认识

这一章不要求你安装软件，也不要求你输入命令。目标只有一个：先看清整套工具如何合作，再决定后面每一步为什么要学。

本章目标

学习完成后，你能够：

- 用自己的话解释 ChatGPT 与 Codex 的区别。
- 判断一项任务应该先交给 ChatGPT 思考，还是交给 Codex 执行。
- 说清 Git、GitHub、VS Code、Windows 与 macOS 在整套流程中的位置。
- 看懂一个需求如何从想法变成 GitHub 上可以查看的项目记录。
- **预计学习时间**：30 ~ 45 分钟。
- **适合读者**：没有编程、AI 或 Git 基础的读者，以及希望把 AI 用于办公、管理或创业的人。
- **需要的前置知识**：无需前置知识。
- **开始前准备**：只需要认真阅读；本章没有安装或修改电脑的操作。

为什么学习这一章？

为什么现在学习 AI？

AI（人工智能）是一类帮助计算机理解内容、生成答案或协助完成任务的技术。过去，使用软件通常要求人学习软件的菜单和规则；现在，人可以先用日常语言说明目标，再由 AI 帮助整理思路或执行部分工作。

这不表示 AI 会自动把所有事情做对。它更像一位速度很快、知识面很广，但仍需要明确任务和人工检查的助手。越早学会正确提出需求、划分任务和检查结果，就越容易把 AI 变成稳定工具，而不是偶尔尝鲜的聊天窗口。

为什么未来越来越重要？

许多工作都包含类似过程：阅读资料、整理信息、制定方案、修改文件、检查结果。AI 正在进入这些环节。真正重要的能力，不只是“会不会问一个问题”，而是能否做到：

1. 把模糊想法说清楚。
2. 把大任务拆成可以检查的小任务。
3. 让合适的工具执行。
4. 判断结果是否符合目标。
5. 保留过程，方便继续修改和与他人合作。

这些能力不会因为某个软件改版就失效。

为什么不仅程序员需要学习？

AI 开发并不只等于编写程序。这里的“开发”，也可以理解为把一个想法逐步做成可使用、可修改、可保存的成果。

- **办公人员**可以让 ChatGPT 帮助整理会议需求，再让 Codex 修改一组规范文档并检查链接。
- **管理人员**可以先明确目标和验收标准，再让执行过程留下清楚记录，减少“做了很多但无法核对”的情况。
- **创业者**可以把产品想法拆成页面、文档和任务清单，用较小成本验证方向。
- **学习者**可以请 ChatGPT 解释概念，再让 Codex 在练习项目中完成实际操作。

解决什么问题？

这一章主要解决三个常见困惑：

- 把 ChatGPT 和 Codex 当成同一个工具，不知道该让谁做什么。
- 把 Git 和 GitHub 当成同一个东西，不知道文件究竟保存在哪里。
- 一上来就安装很多软件，却不知道它们为什么存在、如何连接。

不学习会遇到什么困难？

如果没有整体地图，你可能会直接让 AI 修改重要文件，却没有说明范围；也可能以为“保存在电脑上”就等于“已经上传到 GitHub”。问题通常不是不会点击，而是不知道当前处于哪一步。

所以，本书先讲关系，再讲安装。

一、这是什么？

一句话解释

ChatGPT 主要帮助你把事情想清楚，Codex 主要帮助你在真实项目中把事情做出来。

什么是 ChatGPT？

ChatGPT 是 OpenAI 提供的对话式 AI 产品。所谓“对话式”，是指你可以像交谈一样输入问题、补充条件、要求修改，并根据回答继续追问。它的主要作用是帮助人理解、分析和策划。例如：

- 把一个模糊想法整理成清楚目标。
- 解释不熟悉的概念。
- 比较多个方案的优缺点。
- 把一个大任务拆成较小步骤。
- 检查计划是否遗漏重要条件。

ChatGPT 适合做什么：

- 讨论 “应该做什么”。
- 整理 “为什么要这样做”。
- 起草结构、清单和验收标准。
- 审阅一段已经提供给它的内容。

ChatGPT 不适合做什么：

- 在没有项目访问能力时，声称已经修改了你电脑上的文件。
- 替你决定所有重要事项。
- 在没有核实的情况下保证信息永远正确或最新。
- 在目标和边界都不清楚时直接进行大范围修改。

为什么要了解这些边界？因为 “给出建议” 和 “真正改变项目” 是两件不同的事。把二者混在一起，容易出现你以为已经完成，其实只得到一段说明的情况。

什么是 Codex？

Codex 是 OpenAI 的开发执行助手。这里的 “开发执行” 不只指写程序，也包括阅读项目文件、修改文档、运行检查和汇报结果。与普通对话相比，Codex 的关键特点是它可以在获得允许的项目范围内行动。例如，它可以：

- 找到指定文件并阅读现有内容。
- 按要求修改文档或代码。
- 运行命令检查格式、链接或测试。
- 展示修改了哪些内容。
- 使用 Git 保存版本，并在获得允许时推送到 GitHub。

为什么需要 Codex？因为一个完整任务通常不止需要答案，还需要对真实文件进行操作和验证。ChatGPT 可以帮助你设计目录；Codex 可以在项目中创建或修改这些文件，并检查结果是否符合要求。Codex 也不是无人监督的自动机器。你仍要告诉它：

- 目标是什么。
- 哪些文件可以修改。
- 哪些内容不能碰。
- 完成后怎样才算合格。

ChatGPT 与 Codex 的区别

比较内容	ChatGPT	Codex
主要角色	策划与解释	执行与验证
常见输入	问题、想法、资料	明确任务和项目范围
常见输出	分析、方案、文字内容	文件修改、命令结果、检查报告
是否直接操作项目	通常不直接操作本地项目	在获得权限后可以操作指定项目
最需要人的地方	判断建议是否合理	限定边界并验收修改

这不是谁更高级的问题，而是分工不同。就像建筑师负责把需求变成图纸，施工团队负责按图纸落地；两者必须互相配合。

专业术语解释

术语	中文解释	本章中的作用
AI	人工智能，帮助计算机理解、生成或处理内容的一类技术	ChatGPT 与 Codex 都使用 AI
项目	为完成一个目标放在一起的一组文件和规则	Codex 通常在指定项目中工作
Git	记录文件变化的版本管理工具	保存每次重要修改的记录
GitHub	在线保存 Git 仓库并支持协作的网站	让项目可以在线查看和同步
Repository	中文叫 “仓库”，指项目文件和版本记录的集合	GitHub 上一个项目的基本单位
Commit	中文叫 “提交”，指给一组变化做一次带说明的版本记录	表示一个可追踪的完成节点
VS Code	一种查看、编辑和管理项目文件的软件	人可以在其中查看实际文件
Terminal	中文叫 “终端”，通过文字命令操作电脑的窗口	Git 和开发工具常在这里运行

更多术语可以查看 [全书术语表](#)。

历史背景（简要）

过去，人通常先学习编程语言，再通过文字命令和编辑器开发项目。现在，ChatGPT 和 Codex 让人可以先用自然语言表达目标。自然语言就是日常说话和写作使用的语言，例如中文。工具变得更加容易接近，但项目仍然需要版本记录、在线保存和人工检查。因此，AI 没有让 Git、GitHub 或编辑器消失，而是让普通人更容易进入这套流程。

二、为什么这样设计？

ChatGPT 与 Codex 如何配合？

一项任务可以分成 “想清楚” 和 “做出来” 两个阶段。ChatGPT 负责：

- **分析**：理解问题和限制条件。
- **策划**：提出结构、顺序和可选方案。
- **拆任务**：把大目标变成可以逐项检查的小任务。
- **审核**：根据你提供的结果检查是否符合原目标。

Codex 负责：

- **执行**：在允许的范围内操作真实文件。
- **开发**：创建或调整文档、代码和配置。
- **修改**：根据反馈修正具体内容。
- **测试**：运行检查，确认格式、链接或功能是否正常。

为什么要分工？因为思考阶段允许快速比较方案，执行阶段则必须尊重文件、权限和项目历史。先想清楚再执行，返工更少；执行后再审核，错误更容易被发现。

简化工作原理

1. 用户说明想达到什么结果。
2. ChatGPT 帮助用户把目标和标准说清楚。
3. 用户把明确任务交给 Codex，并限定可修改范围。
4. Codex 读取项目、实施修改并检查结果。
5. Git 记录这次修改。
6. GitHub 保存在线版本。
7. 用户检查最终页面或文件，决定是否继续。

这是入门级解释。真实工作中，步骤可能往返多次，而不是永远只走一遍。

实际应用场景

- **办公**：规划一套新人培训手册，再批量检查章节链接。
- **管理**：把模糊需求改成任务清单和验收标准，再跟踪每次修改。
- **创业**：整理产品想法，建立项目说明和版本路线图，再逐步验证。

三、一步一步操作

本节不要求你亲自操作电脑，而是用 GPT-Codex-Guide 项目演示一次完整流程。

步骤 1：提出需求

- **操作**：用户提出“建立一本面向零基础用户的中文 AI 开发指南”。
- **为什么这样做**：先说清读者和目标，后续工具才知道什么内容算合适。
- **可能遇到的问题**：只说“帮我写一本书”，范围太大，也无法判断完成标准。
- **解决方法**：补充目标读者、主题、文件范围和本阶段不要做什么。
- **完成后的验证方法**：需求中能看出“为谁做、做什么、这次做到哪里”。

步骤 2：策划项目

- **操作**：使用 ChatGPT 讨论章节、学习顺序和每章应该回答的问题。
- **为什么这样做**：先建立地图，可以避免写到一半才发现章节重复或缺失。
- **可能遇到的问题**：方案看起来完整，但用词太专业。
- **解决方法**：明确要求面向零基础，并让每个术语立即解释。
- **完成后的验证方法**：得到一份能按顺序学习、每章目标清楚的目录。

步骤 3：交给 Codex 执行

- **操作**：要求 Codex 只修改指定项目和指定文件，并说明禁止修改的范围。
- **为什么这样做**：Codex 能操作真实文件，所以必须先划定边界。
- **可能遇到的问题**：任务没有写清允许修改哪些文件。
- **解决方法**：像本章任务一样明确写出“只编写 docs/01-整体认识.md”。
- **完成后的验证方法**：Codex 汇报的变更范围与允许范围完全一致。

步骤 4：在 VS Code 或文件列表中检查

- **操作**：打开项目文件，查看标题、段落、表格和流程图。
- **为什么这样做**：AI 完成修改不等于内容已经合格，最终仍需人检查。
- **可能遇到的问题**：只看 Codex 的总结，没有查看实际文件。
- **解决方法**：打开被修改文件，至少检查开头、关键表格、链接和结尾。
- **完成后的验证方法**：文件可以正常打开，内容与任务要求一致。

步骤 5：使用 Git 记录修改

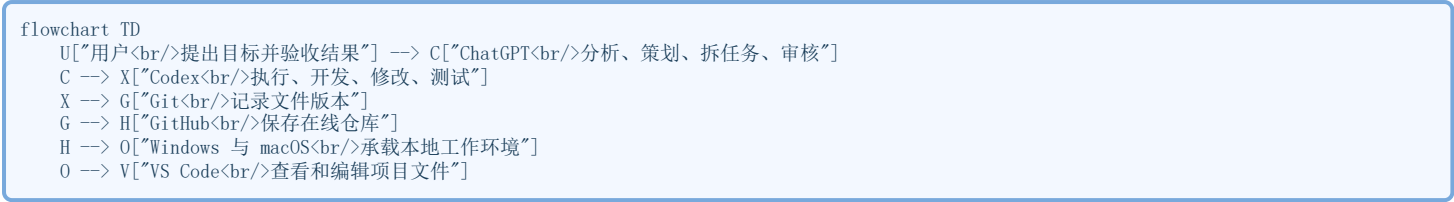
- **操作**：检查变化后创建一次 Commit（提交）。
- **为什么这样做**：提交会把这次变化保存成带说明的版本记录，方便以后查找。
- **可能遇到的问题**：把不相关修改一起提交。
- **解决方法**：提交前查看变更列表，确认只有本次允许的文件。
- **完成后的验证方法**：Git 显示新的提交，并且工作区没有意外变化。

步骤 6：推送到 GitHub

- **操作**：将本地提交 Push（推送）到 GitHub。
- **为什么这样做**：本地提交只在当前电脑上；推送后才有在线副本。
- **可能遇到的问题**：以为创建提交就已经上传。
- **解决方法**：在 GitHub 仓库页面查看最新提交和文件内容。
- **完成后的验证方法**：GitHub 页面显示新的提交说明，第一章可以在线打开。

四、流程图

下面的图展示从用户提出目标，到项目在 GitHub 留下记录的整体关系。



需要特别注意：这张图按照本章的学习叙述排列，不表示 GitHub 运行在 Windows 或 macOS 里面。GitHub 是在线服务；Windows 与 macOS 是电脑的操作系统；VS Code 是安装在操作系统上的编辑器。

简单记忆：

- ChatGPT 帮你想。
- Codex 帮你做。
- Git 帮你记。
- GitHub 帮你在线保存和协作。
- Windows 或 macOS 提供电脑环境。
- VS Code 帮你看见和编辑项目。

五、实际案例

案例 1：办公人员整理操作手册

- **背景：**一位办公人员需要把零散流程整理成新人手册。
- **操作过程：**先用 ChatGPT 规划目录和读者问题，再让 Codex 在指定项目中整理文档并检查链接。
- **结果：**手册结构一致，修改有 Git 记录，并可在 GitHub 查看。
- **总结：**不会编程也可以使用这套流程；关键是把目标和边界说清楚。

案例 2：管理人员建立项目规范

- **背景：**团队文件命名混乱，每个人写法不同。
- **操作过程：**管理者用 ChatGPT 比较规范方案，确认规则后让 Codex 创建规范文件并检查现有引用。
- **结果：**团队拥有统一标准，也能追踪标准何时改变。
- **总结：**AI 不只是生成文字，还能帮助把管理规则落实到真实项目。

案例 3：GPT-Codex-Guide 第一章

- **背景：**项目已有章节大纲，但第一章还不能正式发布。
- **操作过程：**用户限定只修改第一章，Codex 对照统一模板编写内容，检查 Markdown 和 Mermaid，再用 Git 提交并推送到 GitHub。
- **结果：**第一章成为可在线阅读、可继续维护的版本。
- **总结：**从需求到 GitHub 的每一步都有明确责任和验证点。

六、常见错误

编号	错误或现象	错误原因	解决方法	避免方式
1	以为 ChatGPT 已修改电脑文件	把对话回答当成真实执行	打开文件检查是否变化	区分“给建议”和“操作项目”
2	让 Codex 自己猜目标	任务只有一句模糊要求	补充读者、范围和完成标准	使用清楚的任务说明
3	没有限定可修改文件	忽略了执行边界	明确列出允许和禁止范围	每次任务先写边界
4	把 Git 当成 GitHub	两者名字常一起出现	记住 Git 负责记录，GitHub 负责在线保存	用一句话分别解释
5	保存文件后以为已经提交	混淆保存与 Git 提交	查看 Git 状态并创建提交	按“保存、提交、推送”检查
6	提交后以为已经上传	本地提交尚未推送	推送后查看 GitHub	每次在网页复核
7	只看 AI 总结，不看文件	过度相信执行报告	打开实际文件抽查关键位置	把人工验收列为固定步骤
8	一次要求修改太多内容	任务过大，难以检查	拆成多个小任务	一次只完成一个清晰目标
9	遇到术语就跳过	担心打断阅读	立即查看本章解释或术语表	用自己的话复述术语
10	认为不会编程就不能参与	把开发误解为只写代码	从需求、文档和检查开始	先练习可观察的小任务

七、本章练习

练习 1：用一句话区分工具

- **任务：**不用照抄正文，分别用一句话解释 ChatGPT、Codex、Git 和 GitHub。
- **为什么练习：**能用自己的话说明，才表示真正建立了基本概念。
- **完成标准：**四句话没有把“策划、执行、记录、在线保存”混在一起。

练习 2：为真实任务选择工具

- **任务：**阅读下面四件事，并写出主要交给哪个工具：比较两个方案、修改项目文件、记录文件版本、在线查看仓库。
- **为什么练习：**工具地图只有用于判断时才有价值。
- **完成标准：**答案依次能对应 ChatGPT、Codex、Git、GitHub，并能说出原因。

练习 3：设计一次完整流程

- **任务：**选择一个小目标，例如制作个人学习清单，写出从提出需求到 GitHub 复核的六个步骤。只写计划，不需要真的安装或操作。
- **为什么练习：**把多个工具串成流程，是本章最重要的能力。
- **完成标准：**计划包含用户目标、ChatGPT 策划、Codex 执行、人工检查、Git 提交和 GitHub 复核。

八、本章总结

重点回顾

- ChatGPT 主要负责分析、策划、拆任务和审核。
- Codex 主要负责执行、开发、修改和测试。
- Git 记录项目变化，GitHub 保存在线仓库。
- Windows 与 macOS 提供电脑环境，VS Code 帮助人查看和编辑项目。
- AI 可以服务办公、管理、创业和学习，不只服务程序员。
- 无论 AI 多强，清楚的目标、明确的边界和人工验收都不能省略。

完成检查

- ☐ 我能解释 ChatGPT 与 Codex 的区别。
- ☐ 我能解释 Git 与 GitHub 的区别。
- ☐ 我知道保存、提交和推送不是同一件事。
- ☐ 我能说出从需求到 GitHub 的基本流程。
- ☐ 我知道为什么 AI 的结果仍需要人工检查。

下一章预告

下一章将学习“为什么安装这些软件”。

你会进一步区分：哪些工具是网站，哪些需要安装，哪些需要账号，哪些属于必需、推荐或可选。这样做的原因是：先知道每个工具解决什么问题，再安装，才能避免在电脑上堆满自己并不理解的软件。

第二章

第2章 为什么要学习 ChatGPT 与 Codex

这一章不安装软件，也不要求你会编程。我们先回答一个更重要的问题：为什么值得花时间学习这两个工具？

本章目标

学习完成后，你能够：

- 说清 ChatGPT 与 Codex 分别是什么。
- 理解为什么普通办公人员、管理者、创业者和学习者也需要 AI 能力。
- 判断什么时候适合先用 ChatGPT，什么时候需要 Codex。
- 为自己写出一个简单、可执行的学习目标。
- **预计学习时间**：20 ~ 30 分钟。
- **适合读者**：没有 AI、编程或 Git 基础的读者。
- **需要的前置知识**：读完第 1 章即可；没有也可以直接阅读。
- **开始前准备**：无需安装任何软件。

为什么学习这一章？

很多人第一次接触 AI，只把它当成一个“会回答问题的聊天工具”。这没有错，但只看到了第一步。

真正有价值的能力，是把自己的目标说明白，让 AI 帮助分析，再把方案落实到真实文件和项目中。ChatGPT 与 Codex 正好对应这两个阶段：一个帮助思考，一个帮助执行。

如果不先理解为什么学习，你可能会收藏很多提示词、安装很多软件，却不知道它们怎样解决自己的问题。本章的目的，就是让学习有方向。

一、ChatGPT

① 这是什么？

ChatGPT 是一种可以用日常语言交流的 AI 工具。你可以像向一位助手说明情况一样，输入问题、补充条件、要求它解释或重新整理。“日常语言”就是我们平时说话和写字使用的语言，例如中文。你不需要先学会编程，才能开始与 ChatGPT 对话。

② 为什么要学习？

工作和学习中，最难的部分常常不是点击按钮，而是：

- 不知道问题应该怎样描述。
- 脑中有想法，却整理不成清楚计划。
- 资料很多，不知道先看什么。
- 写出内容后，不知道是否遗漏重点。

学习 ChatGPT，是在练习如何把模糊想法变成清楚问题。这个能力不仅用于 AI，也会改善沟通、写作和计划。

③ 有什么作用？

ChatGPT 常用于：

- 解释陌生概念。
- 整理会议记录或资料。
- 比较不同方案。
- 拆分大任务。
- 起草文档结构。
- 根据标准帮助检查内容。

它不适合替你做最终决定，也不能保证每次回答都正确。为什么？因为 AI 是根据你提供的信息生成回答；如果信息不完整，答案也可能偏离目标。因此，重要内容仍要由人核对。

④ 学完以后能做什么？

学会基础使用后，你可以：

- 写出更清楚的问题。
- 让 ChatGPT 用零基础语言重新解释。
- 把一个大目标拆成小步骤。
- 要求回答符合指定格式。
- 发现回答不合适时，补充条件并继续修改。

二、Codex

① 这是什么？

Codex 是 OpenAI 的开发执行助手。它可以在获得允许的范围内读取项目文件、修改内容、运行检查并汇报结果。这里的“开发”不只指写程序。整理一组文档、修正链接、建立项目规范，也属于把想法落实成成果的过程。

② 为什么要学习？

ChatGPT 可以告诉你“应该怎样做”，但真实工作还需要打开文件、修改内容、检查结果和保存版本。

如果每个步骤都完全靠手工完成，任务一多就容易遗漏。Codex 的价值，是在明确边界后帮助执行这些步骤，同时把过程展示出来供人检查。

学习 Codex，不是为了把控制权交给 AI，而是为了学会怎样安全地委派任务。所谓“委派”，就是把一项工作交给助手执行，同时明确目标、范围和验收标准。

③ 有什么作用？

Codex 常用于：

- 阅读项目现状。
- 按要求创建或修改文件。
- 检查 Markdown、链接或程序测试。
- 比较修改前后的差异。
- 使用 Git 保存版本记录。
- 在获得允许后，把结果推送到 GitHub。

Codex 不应该在范围不清楚时随意修改整个项目。为什么？因为执行工具会接触真实文件；边界越清楚，结果越容易检查，也越安全。

④ 学完以后能做什么？

学会基础使用后，你可以：

- 明确告诉 Codex 只修改哪些文件。
- 写出“允许做什么、禁止做什么”。
- 要求它先检查现状再执行。
- 查看修改内容和检查结果。
- 判断任务是否真正完成，而不是只看一句总结。

三、为什么要同时学习？

① 这是什么？

同时学习，是指把 ChatGPT 的策划能力与 Codex 的执行能力连接起来，而不是把它们当成两个互不相关的工具。

② 为什么要学习？

只有策划，方案可能一直停留在对话里；只有执行，任务可能因为目标不清而反复修改。先想清楚，再动手；执行以后，再检查。这是比“让 AI 随便做一下”更稳定的方法。

③ 有什么作用？

两者配合时，可以形成一个简单循环：

1. 用户提出目标。
2. ChatGPT 帮助分析和拆分任务。
3. 用户确认范围和完成标准。
4. Codex 在项目中执行。
5. 用户检查结果。
6. 不合适就补充要求，再进行下一轮。

④ 学完以后能做什么？

你可以把一个真实需求写成两部分：

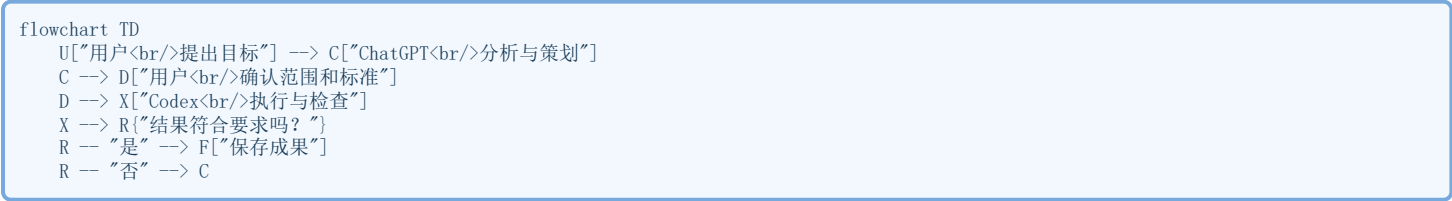
- **策划任务**：请 ChatGPT 帮助澄清目标、对象、步骤和标准。
- **执行任务**：请 Codex 在指定范围内修改文件并运行检查。

这样做的好处是每一步都能看见、能解释、能验证。

四、工作关系图

下面的流程图说明从想法到成果的基本顺序。

流程图



为什么流程会返回 ChatGPT？因为第一次方案不一定完美。发现问题后重新分析，再交给 Codex 修改，是正常工作过程，不代表失败。

五、实际场景

场景 1：办公人员整理制度

- **这是什么**：把零散制度整理成统一文档。
- **为什么学习**：手工复制容易遗漏或格式不一致。
- **有什么作用**：ChatGPT 先规划结构，Codex 再整理文件和检查链接。
- **学完能做什么**：写出清楚要求，并检查整理结果。

场景 2：管理者制定项目计划

- **这是什么：**把目标、任务和完成标准写成计划。
- **为什么学习：**只有一句口头要求，团队容易理解不同。
- **有什么作用：**ChatGPT 帮助拆任务，Codex 把确认后的计划落实到项目文档。
- **学完能做什么：**把模糊要求变成可检查的任务。

场景 3：创业者验证想法

- **这是什么：**先用较小成果判断一个想法是否值得继续。
- **为什么学习：**直接投入大量时间和资金，风险较高。
- **有什么作用：**ChatGPT 帮助分析用户问题，Codex 帮助制作说明文档或简单项目。
- **学完能做什么：**从一个小目标开始，完成后再决定下一步。

六、常见误区

| 误区 | 为什么不准确 | 正确做法 |

|---|---|---|

| AI 只适合程序员 | 分析、写作、整理和检查也属于常见工作 | 从自己的真实小任务开始 |

| ChatGPT 和 Codex 是同一个工具 | 两者都使用 AI，但主要分工不同 | 记住“ChatGPT 策划，Codex 执行” |

| AI 给出的内容一定正确 | 回答可能受信息不足或理解偏差影响 | 检查重要事实和最终结果 |

| 需求越短越省事 | 过于模糊会让结果偏离目标 | 写清对象、范围和完成标准 |

| Codex 会自动知道哪些文件不能改 | 执行边界需要用户明确说明 | 列出允许和禁止修改的范围 |

| 学 AI 必须先学编程 | 日常语言就是学习入口 | 先学习提问、拆任务和验收 |

| 一次要 AI 完成全部工作最快 | 大任务难以检查，也容易返工 | 拆成小任务逐步完成 |

| AI 可以替代人的判断 | AI 不承担你的目标和后果 | 由人确认重要决定 |

| 看见总结就等于任务完成 | 总结可能与实际文件不一致 | 打开文件并查看检查结果 |

| 学会几个提示词就足够 | 工具和场景会变化 | 学习目标、边界、执行和验证的方法 |

七、本章练习

练习 1：找到一个真实问题

写下一件最近让你重复花时间的小事，例如整理会议记录。
完成标准：用一句话说清“谁遇到了什么问题”。

练习 2：分成策划与执行

把练习 1 分成两项：

- ChatGPT 应该帮助你搞清楚什么？
- Codex 应该在什么文件范围内做什么？

完成标准：两项任务没有混淆“搞清楚”和“做出来”。

练习 3：补充验收标准

为执行任务写出三个检查点，例如文件能打开、标题完整、没有修改其他文件。
完成标准：别人只看检查点，也能判断任务是否完成。

八、本章总结

【本章总结】
ChatGPT 帮助你理解问题、整理想法和规划任务；Codex 帮助你在真实项目中执行、修改和检查。
学习它们的真正意义，不是追赶一个新名词，而是建立一套可重复的方法：

1. 把目标说清楚。
2. 把任务拆小。
3. 让合适的工具执行。
4. 由人检查结果。
5. 根据结果继续改进。

你不需要先成为程序员。先从小而真实的问题开始，就已经进入了 AI 协作的第一步。

下一章预告

【下一章预告】
下一章将进入 Windows 安装。你会学习需要准备哪些工具、从哪里安全下载，以及每一步安装完成后怎样确认结果。
如果你使用 macOS，可以先了解安装原则，再阅读对应的 Mac 安装章节。

第3章 ChatGPT 是什么

ChatGPT 不是“什么都知道的机器”，也不是会自动替你完成所有工作的员工。正确认识它，才能放心地把它用在学习、办公和日常思考中。

本章目标

学习完成后，你能够：

- 用一句简单的话解释 ChatGPT。
- 判断哪些任务适合交给 ChatGPT 协助。
- 识别 ChatGPT 的能力边界。
- 写出包含目标、背景、要求和输出形式的清楚问题。
- **预计学习时间**：20 ~ 30 分钟。
- **适合读者**：第一次接触 ChatGPT，或已经使用但不知道怎样稳定获得好结果的人。
- **需要的前置知识**：无需编程知识。
- **开始前准备**：本章以理解和练习为主，不要求安装软件。

一、ChatGPT 是什么？

一句话解释

ChatGPT 是一种可以通过对话帮助你理解、整理和生成内容的 AI 工具。
AI 是“人工智能”的简称。它让计算机能够处理文字、图片等信息，并根据你的要求给出结果。

它怎样与你交流？

你在对话框中输入一段话，ChatGPT 阅读后给出回答。你可以继续补充条件、指出问题或要求重写。前后多轮交流合在一起，称为“对话”。你输入的要求常被称为“提示词”。这个词听起来专业，其实可以把它理解为“你交给 AI 的任务说明”。

例如：

请用零基础读者能听懂的语言，解释 Git 与 GitHub 的区别。先各用一句话说明，再用一个生活比喻比较。

这只比输入“介绍 Git”更清楚。为什么？因为它说明了读者是谁、要解释什么、希望怎样输出。

ChatGPT 是怎样回答的？

入门阶段不需要理解内部技术。你只要记住：ChatGPT 会根据你的输入和对话内容生成回答。
“生成”不等于“从一本永远正确的答案册中复制”。它可能理解错误，也可能给出听起来合理、实际却不准确的内容。因此，ChatGPT 适合协助思考，但重要事实仍要核对。

学完这一节能做什么？

你可以向别人解释：

- ChatGPT 是对话式 AI 工具。
- 提示词就是任务说明。
- 回答可以继续修改，不必一次问到完美。
- AI 生成的结果需要人判断。

二、ChatGPT 能做什么？

ChatGPT 最擅长处理“用语言表达的任务”。它可以加快理解和整理，但最终目标仍由你决定。

1. 解释知识

它可以把陌生概念换成更简单的说法，也可以使用比喻、例子或分步骤说明。
为什么有用？因为传统资料往往默认读者已经懂一些基础知识。你可以要求 ChatGPT 从零开始解释，并随时追问不明白的部分。
学完以后，你可以要求：

请不要使用复杂术语。每出现一个新词，立即用一句中文解释。

2. 整理信息

它可以帮助整理会议记录、长段文字、想法清单或学习笔记。
为什么有用？因为信息杂乱时，人很难看出重点。先整理出主题、结论和待办事项，再由人核对，会更容易继续工作。
注意：如果原始资料包含姓名、密码、公司机密或其他敏感信息，不要直接放入对话。敏感信息就是不应该公开、可能影响个人或组织安全的内容。

3. 规划任务

它可以把“我要做一个项目”拆成较小步骤，帮助你看到先后顺序和检查点。
为什么有用？因为任务太小时，人容易不知道从哪里开始。小步骤更容易完成，也更容易发现问题。
例如，建立本指南时，可以先拆成：确定读者、规划章节、建立模板、逐章编写、检查和发布。

4. 起草内容

它可以起草邮件、说明、清单、表格、文章结构或方案初稿。
“初稿”是第一次形成的版本，还需要检查和修改。为什么要强调初稿？因为把 AI 输出直接当成最终稿，容易保留错误、空话或不合适语气。

5. 比较和检查

它可以按照你提供的标准比较方案，或检查内容是否遗漏要求。
为什么有用？因为明确标准后，检查会更有方向。例如：“面向零基础、全部中文、每个术语立即解释”就是可以逐项核对的标准。

学完这一节能做什么？

你可以把 ChatGPT 用于五类任务：解释、整理、规划、起草和检查。遇到新任务时，先判断它属于哪一类，再提出要求。

三、ChatGPT 不能做什么？

了解限制不是为了否定 ChatGPT，而是为了安全、稳定地使用它。

1. 不能保证每次都正确

ChatGPT 可能误解问题，也可能生成错误信息。文字流畅不等于事实正确。
正确做法：

- 重要事实查看可靠来源。
- 数字、日期、法律、医疗和财务信息要特别核实。
- 不确定时要求它明确说明不确定之处。

2. 不能替你承担决定和责任

它可以列出选择和风险，但不能替你承担结果。
为什么？因为只有你知道完整背景、真实目标和可以接受的风险。涉及工作、人事、合同、健康或资金时，最终决定应由合适的人作出。

3. 不能自动知道你没有说出的背景

如果你只说“帮我写一个方案”，它不知道方案给谁看、解决什么问题、何时完成。
正确做法是补充必要背景，但不要提供不需要的隐私或机密。

4. 不能默认操作你的本地项目

普通对话中的 ChatGPT 可以给出建议或内容，但这不代表它已经修改你电脑上的文件。
为什么要区分？“回答怎样做”和“真实执行修改”是两件事。需要在项目中读取、修改和检查文件时，应使用获得相应权限的 Codex，并明确操作范围。

5. 不能代替人工检查

AI 输出可能有错别字、遗漏、重复或不合适的表达。发布、发送或执行之前，必须由人检查。

学完这一节能做什么？

你可以识别五条边界：不保证正确、不替人负责、不知道未提供的背景、不等于已经操作文件、不能省略人工检查。

四、怎样正确使用 ChatGPT？

一个好问题不需要华丽术语，只要包含四项信息：

1. **目标**：你想完成什么。
2. **背景**：为什么做，给谁使用。
3. **要求**：必须包含什么，不能出现什么。
4. **输出形式**：希望得到清单、表格、步骤还是短文。

第一步：先说目标

不要只写“帮我看看”。
可以写：

我想为完全没有 AI 基础的同事准备一份 ChatGPT 入门说明。

为什么这样做？目标越清楚，回答越容易围绕真正问题展开。

第二步：补充必要背景

可以补充：

读者主要做办公室工作，没有编程经验。

为什么这样做？同一个主题，写给程序员和写给零基础办公人员，解释方式完全不同。

第三步：列出要求和边界

可以写：

全部使用中文，不使用复杂术语；每个新词立即解释；不要包含注册和付费说明。

为什么这样做？要求告诉 ChatGPT 什么叫“合格”，边界防止回答偏离范围。

第四步：指定输出形式

可以写：

先用一句话解释，再列出三个用途和三个限制，最后给一个练习。

为什么这样做？输出形式明确后，内容更容易阅读和检查。

第五步：检查并继续修改

收到回答后，问自己：

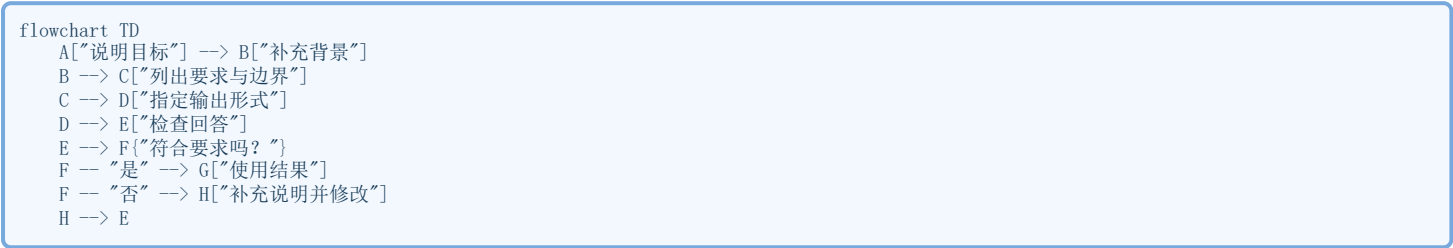
- 是否回答了真正问题？
- 是否有我不理解的词？
- 是否有事实需要核对？
- 是否符合长度和格式要求？
- 是否包含不应该出现的信息？

如果不满意，不必重新开始。可以继续说：

第二部分太难，请保留意思，改成零基础读者能懂的短句。

这就是正确使用 ChatGPT 的核心循环：

流程图



一个完整示例

我想理解 GitHub，读者是完全没有编程经验的办公人员。请全部使用中文，先用一句话解释它是什么，再说明三个用途和两个不能做的事情。不要介绍安装步骤。最后设计一个不用注册账号也能完成的小练习。

这个任务说明并不复杂，但目标、背景、要求和输出形式都很清楚。

五、本章练习

练习 1：判断是否适合

下面哪些任务适合请 ChatGPT 协助？

- 把一段会议记录整理成待办事项。
- 保证一份合同在法律上绝对没有风险。
- 用简单语言解释一个陌生概念。
- 自动知道你电脑里哪个文件最重要。

完成标准：你能指出第一项和第三项适合；第二项需要专业人员判断，第四项缺少访问和背景。

练习 2：补全问题

把“帮我写个介绍”补充成一个完整任务，至少写出目标、背景、要求和输出形式。

完成标准：别人只看你的任务说明，也知道写给谁、写什么和怎样判断结果。

练习 3：完成一次修改对话

先提出一个简单问题，再根据第一次回答补充一条新要求，例如“缩短一半”或“解释所有术语”。

完成标准：第二次回答比第一次更接近你的真实需要。

六、本章总结

【本章总结】

ChatGPT 是一种通过对话帮助人理解、整理和生成内容的 AI 工具。它能帮助解释知识、整理信息、规划任务、起草内容和按照标准检查结果。但它不能保证每次正确，不能替人承担决定，也不能自动知道未说明的背景。正确使用 ChatGPT，不是寻找一句神奇提示词，而是把任务说清楚：

1. 说明目标。
2. 补充必要背景。
3. 列出要求和边界。
4. 指定输出形式。
5. 检查结果并继续修改。

你始终是目标的决定者和结果的验收者。

下一章预告

【下一章预告】

《第四章：Codex 是什么》

第四章

第4章 Codex 是什么

ChatGPT 可以帮助你把事情想清楚，Codex 可以在获得允许后进入项目，把明确任务落实到真实文件中。理解这种区别，是安全使用 Codex 的第一步。

本章目标

学习完成后，你能够：

- 用一句简单的话解释 Codex。
- 说清 Codex 与 ChatGPT 的主要区别。
- 判断哪些任务适合使用 Codex。
- 为 Codex 写出包含目标、范围和验收标准的任务。
- 知道执行完成后为什么仍要人工检查。
- **预计学习时间**：20 ~ 30 分钟。
- **适合读者**：没有编程基础，但希望让 AI 协助处理真实项目的人。
- **需要的前置知识**：理解 ChatGPT 的基本作用。
- **开始前准备**：本章不要求安装或运行 Codex。

一、Codex 是什么？

一句话解释

Codex 是一个可以在授权范围内阅读项目、修改文件、运行检查并汇报结果的开发执行助手。

什么是“开发执行助手”？

“开发”不只显示写程序。把一套文档整理成统一格式、修复链接、更新目录、生成 PDF，也是在把想法变成可以使用的成果。“执行助手”表示它不只是告诉你应该怎样做，还可以在获得权限后实际完成步骤。例如：

- 打开指定项目中的文件。
- 根据要求修改内容。
- 运行检查命令。
- 比较修改前后的差异。
- 使用 Git 保存一次版本记录。
- 在获得允许时推送到 GitHub。

Git 是记录文件变化的工具。GitHub 是在线保存 Git 仓库并支持协作的网站。“仓库”可以简单理解为项目文件和版本记录的集合。

为什么需要授权范围？

Codex 会接触真实文件，所以需要知道它可以在哪里工作、可以修改什么。例如：

只修改第四章、README 中的第四章目录项和整本 PDF，不要修改已经完成的章节。

这就是清楚的授权范围。为什么重要？因为执行范围越明确，越容易避免误改，也越容易在完成检查。

Codex 怎样工作？

可以把它理解成一个循环：

1. 阅读任务要求。
2. 检查项目现状。
3. 制定执行步骤。
4. 修改文件或运行命令。
5. 检查结果。
6. 向用户汇报。
7. 根据反馈继续修正。

这个过程并不表示 Codex 永远不会出错。它仍可能误解要求、遗漏细节或遇到电脑权限限制。因此，人必须保留最终决定权。

二、Codex 与 ChatGPT 有什么区别？

ChatGPT 与 Codex 都使用 AI，但主要工作位置不同。

比较内容	ChatGPT	Codex
主要作用	分析、解释和策划	执行、修改和检查
常见工作位置	对话内容	获得授权的真实项目
常见结果	方案、说明、清单或初稿	文件变化、检查结果和版本记录
是否需要项目范围	讨论概念时通常不需要	执行前必须明确
人的主要责任	判断建议是否合理	限定权限并验收结果

一个简单比喻

如果要装修一个房间：

- ChatGPT 像帮助你整理需求和比较方案的设计顾问。

- Codex 像按照确认方案进入现场工作的执行助手。
- 你仍然是决定目标、批准范围和验收结果的人。

为什么不能只用其中一个？

只有 ChatGPT，方案可能停留在对话中，没有落实到真实文件。只有 Codex，如果目标没有想清楚，执行速度越快，返工也可能越快。

对话回答不等于项目修改

ChatGPT 在对话中写出一段新内容，不代表电脑上的文件已经变化。Codex 汇报“已经修改”，也不代表你可以跳过检查。

正确做法是：

- 对话阶段检查方案。
- 执行阶段查看实际文件。
- 提交前检查修改范围。
- 推送后在 GitHub 再次确认。

三、Codex 能做什么？

Codex 适合目标清楚、范围明确、结果可以检查的任务。

1. 阅读和理解项目

Codex 可以先查看目录、文件和已有规范，再决定怎样执行。

为什么要先检查？因为项目可能已经有命名规则、模板或未完成的修改。不了解现状就直接动手，容易重复创建文件或破坏原有结构。

2. 修改文档和代码

Codex 可以修改指定的 Markdown 文档、程序代码或配置文件。

Markdown 是一种使用简单符号排版文字的格式，文件通常以 .md 结尾。本书的章节就是 Markdown 文件。

为什么有用？因为它可以把同一要求稳定地应用到明确范围内，例如统一标题、修复链接或补充缺失栏目。

3. 运行检查

修改以后，Codex 可以检查：

- Markdown 标题和链接是否正常。
- Mermaid 流程图围栏是否完整。
- 程序测试是否通过。
- PDF 是否成功生成。
- Git 是否只包含允许的修改。

Mermaid 是一种用文字描述流程图的格式。它让流程图可以跟随文档一起保存和修改。

为什么检查很重要？“文件已经保存”和“结果符合要求”不是同一件事。检查会提供可以观察的证据。

4. 使用 Git 记录变化

Codex 可以查看 Git 状态、创建 Commit（提交）并推送到 GitHub。

Commit 是给一组文件变化建立的版本记录，通常带有一句说明。为什么要提交？因为以后可以知道什么时候改了什么，也更容易发现问题来自哪一次变化。

5. 汇报执行结果

Codex 可以列出修改文件、检查结果和提交编号。

提交编号常称为 Commit Hash，它是一串用于唯一识别某次提交的字符。你不需要记住整串内容，但可以用它找到准确版本。

学完这一节能做什么？

你可以把适合 Codex 的任务归纳为五类：阅读项目、修改文件、运行检查、记录版本和汇报结果。

四、Codex 不能做什么？

1. 不能替你决定目标

如果你只说“把项目做好”，Codex 不知道什么叫“好”，也不知道这次应该做到哪里。

正确做法：说明目标读者、允许范围、完成标准和禁止事项。

2. 不能保证每次理解都正确

Codex 可能把一句含糊要求理解成另一种意思。

正确做法：一次只完成一个目标，重要边界使用清楚、直接的句子。

3. 不能绕过权限和安全限制

Codex 只能在获得允许的范围内操作。访问受保护文件、使用网络或改变外部系统，可能需要单独授权。

为什么这是好事？权限限制可以防止一个普通任务意外影响无关文件、账号或服务。

4. 不能保证所有命令都成功

电脑可能缺少软件、网络可能不可用，文件也可能被其他程序占用。

正确做法：保留完整错误信息，先判断失败原因，再选择低风险处理方式。不要为了让命令成功而盲目关闭安全保护。

5. 不能省略人工验收

即使检查全部通过，内容是否真正适合读者，仍需要人判断。
例如，Markdown 链接正常，只能证明链接能找到文件；它不能证明解释一定通俗、准确或有帮助。

6. 不能在未授权时扩大任务

“编写第四章” 不等于可以顺便重写前面三章。
正确做法：发现相关问题时先报告，不得擅自扩大修改范围。

五、什么时候应该使用 Codex？

可以使用下面五个问题判断：

- 1. 任务是否涉及真实项目文件？
- 2. 目标是否能够清楚说明？
- 3. 允许修改的范围是否明确？
- 4. 完成结果是否可以检查？
- 5. 你是否愿意查看并验收结果？

如果大多数答案是“是”，这项任务通常适合 Codex。

适合使用的情况

- 只修改一个指定章节。
- 检查项目中的内部链接。
- 按现有规范更新 README 目录。
- 运行测试并解释失败原因。
- 重新生成 PDF 并验证文件。
- 检查 Git 修改范围后创建一次提交。

暂时不适合直接执行的情况

- 目标只有“随便优化一下”。
- 不知道哪些文件可以修改。
- 涉及重要数据，却没有备份。
- 需要作出法律、医疗、财务或人事决定。
- 无法说明怎样判断完成。

遇到这些情况，应先用 ChatGPT 或与相关人员讨论，把目标和边界说清楚，再决定是否交给 Codex。

一份简单任务说明

目标：完成第四章《Codex 是什么》。
范围：只修改第四章、README 对应目录项和整本 PDF。
禁止：不要修改已经完成的章节，不要提前开发第五章。
检查：Markdown 正常、PDF 正常、Git 只有允许的文件。
提交：所有变化放在一个 Commit 中并推送到 GitHub。

为什么这份说明有效？因为它同时回答了做什么、在哪里做、不能做什么、怎样验收和怎样保存。

六、为什么采用 “ChatGPT 策划，Codex 执行” ？

GPT-Codex-Guide 不是为了尽快生成大量文字，而是为了验证一套简单、稳定、可重复的协作流程。
ChatGPT 负责：

- 理解用户目标。
- 讨论章节重点。
- 帮助拆分任务。
- 提前发现范围冲突。
- 根据完成标准参与审核。

Codex 负责：

- 检查项目现状。
- 修改指定文件。
- 生成 PDF。
- 检查 Markdown 和 Git。
- 创建提交并推送到 GitHub。

用户负责：

- 决定目标。
- 批准权限。
- 验收内容。
- 决定是否进入下一章。

这种分工的价值不在于名称，而在于把思考、执行和验收分开。每个阶段都有明确责任，也都有检查点。

流程图



```
C --> A["用户<br/>确认计划"]
A --> X["Codex<br/>执行、修改、检查"]
X --> V["用户<br/>验收结果"]
V --> Q{"符合要求吗?"}
Q -- "是" --> G["Git 与 GitHub<br/>记录并保存"]
Q -- "否" --> C
```

为什么一次只开发一章？

- 范围小，要求更容易说清楚。
- 修改少，人工检查更容易。
- 出现问题时，更容易找到原因。
- 每次提交都有单一目的。
- PDF 每章更新一次，可以验证整个发布流程。

这就是本项目希望验证的协作模式：不是追求一次做很多，而是每次都能稳定完成一个清楚目标。

七、本章练习

练习 1：判断工具

下面任务主要适合 ChatGPT 还是 Codex？

- 比较两个章节结构方案。
- 修改项目中的指定 Markdown 文件。
- 解释一个陌生概念。
- 运行链接检查并报告结果。

完成标准：你能把分析与解释交给 ChatGPT，把文件修改与项目检查交给 Codex。

练习 2：补充执行边界

把“请帮我修改项目”改成一份包含目标、允许范围、禁止事项和完成标准的任务。

完成标准：另一个人只看任务说明，就能判断哪些文件可以改。

练习 3：设计验收方法

为“更新 README 目录”写出三个验收点。

参考方向：章节名称是否正确、链接是否存在、是否误改其他目录项。

完成标准：每个验收点都可以通过查看文件或运行检查得到明确结果。

八、本章总结

【本章总结】

Codex 是在授权范围内阅读项目、修改文件、运行检查并汇报结果的开发执行助手。

它与 ChatGPT 的主要区别是：

- ChatGPT 更适合分析、解释和策划。
- Codex 更适合进入真实项目执行和验证。
- 用户始终负责目标、权限和最终验收。

正确使用 Codex，需要做到：

1. 一次只给一个清楚目标。
2. 明确允许和禁止修改的范围。
3. 说明怎样判断任务完成。
4. 查看实际文件和检查结果。
5. 用 Git 与 GitHub 保存可追踪的版本。

“ChatGPT 策划，Codex 执行”并不是把工作完全交给 AI，而是让思考、执行和验收各有位置。

下一章预告

【下一章预告】

《第五章：GitHub 是什么》

第五章

第5章 GitHub 是什么

GitHub 不是安装在电脑里的“另一个 Git”，而是一个在线保存项目、展示版本和开展协作的平台。先理解它的作用，再学习具体操作，会容易得多。

本章目标

学习完成后，你能够：

- 用一句简单的话解释 GitHub。
- 说清 GitHub 与 Git 的区别。
- 理解项目为什么需要在线仓库。
- 说明 GitHub 在 GPT-Codex-Guide 中的作用。
- 识别新手第一次使用 GitHub 时的常见问题。

- **预计学习时间**：20 ~ 30 分钟。
- **适合读者**：没有 Git、GitHub 或编程基础的读者。
- **需要的前置知识**：知道项目是一组为同一目标服务的文件。
- **开始前准备**：本章不要求注册账号，也不要求输入命令。

一、GitHub 是什么？

一句话解释

GitHub 是一个在线保存项目版本、展示内容并支持多人协作的平台。

什么是“在线平台”？

在线平台是通过互联网提供服务的网站。你可以使用账号访问它，不必把 GitHub 本身理解成一个普通文件夹。在 GitHub 中，一个项目通常放在 Repository（仓库）里。Repository 中文叫“仓库”，它包含项目文件以及这些文件的版本记录。例如，GPT-Codex-Guide 仓库中包含：

- README 项目首页。
- docs 目录中的各章文档。
- images、examples 和 templates 等目录。
- PDF 版本。
- 每次重要修改的提交记录。

GitHub 是网络硬盘吗？

它有在线保存文件的作用，但不只是网络硬盘。普通网络硬盘主要关心“现在有哪些文件”。GitHub 还关心：

- 文件以前是什么样。
- 谁在什么时候修改过。
- 为什么进行这次修改。
- 哪些修改已经被项目接受。
- 团队正在讨论哪些问题。

因此，GitHub 更像“带历史记录和协作流程的项目空间”。

为什么要先理解仓库？

如果把仓库只看成一个普通文件夹，就容易不明白提交、分支和同步的意义。“提交”是给一组变化建立一次带说明的版本记录。“分支”是从某个版本延伸出的独立工作线。这些概念会在后续章节逐步学习，本章只需要先知道：GitHub 保存的不只是文件，也保存项目变化的过程。

二、GitHub 与 Git 有什么区别？

Git 与 GitHub 经常一起出现，所以新手很容易把它们当成同一个工具。

比较内容	Git	GitHub
它是什么	记录文件变化的版本管理工具	在线保存 Git 仓库并支持协作的平台
主要位置	可以在自己的电脑上工作	通过互联网提供服务
没有网络能否使用	可以记录本地版本	无法进行在线同步
主要作用	提交、比较和管理版本	备份、分享、讨论和协作
是否必须一起使用	不必	GitHub 主要围绕 Git 仓库工作

“版本管理”就是记录文件怎样一步步变化，方便查看历史或找回较早内容。

一个生活比喻

可以把 Git 想成一本放在自己桌上的项目记录册。每次完成一个阶段，你就在记录册中写下“这次改了什么”。GitHub 像一个在线项目档案室。你把记录册中的新内容同步过去，其他获得权限的人就能查看和协作。这个比喻并不完全等同于真实技术，但能帮助记忆：

- Git 负责记录。
- GitHub 负责在线保存和协作。

为什么本章不讲 Git 命令？

因为先理解 “为什么需要记录和同步” ，再学习命令，才不会把命令当成没有意义的字符。
下一章会专门解释 Git 是什么。本章只建立 GitHub 的整体认识。

三、为什么开发者通常会使用 GitHub？

这里的“开发者”不只指程序员。凡是持续建设文档、网站、程序或其他数字项目的人，都可以参与开发。
GitHub 不是法律规定的必需工具，但它已成为非常常见的项目协作方式。学习它有以下价值。

1. 项目不会只留在一台电脑上

如果文件只保存在电脑里，电脑损坏、文件误删或换设备时，项目可能丢失。
把提交推送到 GitHub，可以拥有在线副本。“推送”是把电脑中的新提交发送到远程仓库。
为什么重要？因为本地保存和在线保存是两层保护。不过，GitHub 也不应被当成所有资料的唯一备份，重要内容仍应有合适的备份方案。

2. 可以清楚展示项目

GitHub 仓库有固定地址，可以展示 README、目录、文件和提交历史。
为什么重要？别人不需要收到一堆散乱附件，就能看到项目当前状态和说明。

3. 可以多人协作

团队成员可以在同一个仓库中工作，并通过 Issue 和 Pull Request 交流。
Issue 中文可理解为“问题或任务记录”。Pull Request 常简称 PR，中文叫“拉取请求”，表示请求维护者检查并合并一组修改。
为什么有用？讨论与修改记录放在项目旁边，比散落在聊天记录中更容易追踪。

4. 可以保留清楚历史

当项目出现问题时，可以查看最近哪些内容发生变化。
为什么重要？没有历史记录时，人只能猜测“以前是不是好的”。有提交记录后，可以定位到具体版本。

5. 可以在不同设备间继续工作

在 Windows 上完成并推送后，可以在 macOS 上取得最新版本继续工作。
为什么重要？GitHub 充当共同的远程仓库，让不同电脑围绕同一份版本历史工作。

四、GitHub 能解决什么问题？

常见问题	GitHub 提供的帮助	仍需注意
文件只在一台电脑上	保存在线仓库	重要资料仍要有备份
不知道改过什么	展示提交历史和差异	提交说明要写清楚
多人互相覆盖文件	提供分支和合并流程	同步前仍要检查状态
问题散落在聊天中	使用 Issue 记录	标题和描述要明确
不知道哪个版本可用	通过主分支和发布记录管理	团队要约定发布规则
换电脑无法继续	从远程仓库取得项目	开始工作前先同步
项目难以展示	README 和仓库页面提供入口	不要公开敏感信息

GitHub 能改善项目管理，但不能自动解决所有问题。
如果提交说明含糊、权限设置错误或团队不遵守流程，仓库仍然会混乱。工具提供能力，真正稳定还需要清楚规则 and 良好习惯。

五、GitHub 在本项目中起什么作用？

GPT-Codex-Guide 的主要目标，是验证 ChatGPT、Codex、GitHub、Windows 与 macOS 的协同开发流程是否简单、稳定、可重复。
GitHub 在这套流程中承担五项作用。

1. 保存在线仓库

每章完成后，本地修改会通过 Git 建立提交，再推送到 GitHub。
为什么这样做？本地文件与在线版本可以互相核对，也可以从其他电脑继续项目。

2. 展示项目首页

README 是仓库首页说明文件。读者进入项目后，可以看到项目目的、学习路线和章节入口。
为什么重要？一个没有首页说明的仓库，就像一本没有封面和目录的书。

3. 保存每章的提交记录

本项目坚持 “一次只开发一章，一次只提交一个 Commit” 。
Commit 就是提交。每章拥有单一目的的提交，出现问题时更容易知道哪一次变化需要检查。

4. 验证协作流程

- 本项目使用：
- 1. 用户确定目标。
 - 2. ChatGPT 参与策划。
 - 3. Codex 修改和检查文件。
 - 4. Git 记录版本。

5. GitHub 保存结果。
6. 用户进行人工验收。

每完成一章，就完整走一遍流程。为什么重复？因为只有多次稳定成功，才能证明流程可重复，而不只是偶然成功一次。

5. 提供公开学习入口

读者可以在线查看 Markdown 文档，也可以取得最新版 PDF。
为什么有用？同一项目同时保留便于维护的源文档和便于阅读的成品文件。
下面是本项目从本地修改到在线保存的简化流程：

流程图

```
flowchart TD
    A["Codex 修改指定文件"] --> B["检查 Markdown 与 PDF"]
    B --> C["Git 创建一次提交"]
    C --> D["推送到 GitHub"]
    D --> E["GitHub 展示最新版本"]
    E --> F["用户人工验收"]
```

六、新手第一次使用 GitHub 容易遇到什么问题？

1. 把 GitHub 当成 Git

原因：两个名称经常一起出现。

解决方法：记住“Git 在本地记录，GitHub 在线保存和协作”。

2. 不知道仓库是否公开

Public 表示公开，Private 表示私有。公开仓库通常可以被他人看到，私有仓库只对获得权限的人开放。

解决方法：创建仓库和上传内容前，检查可见性，不把密码、令牌或公司机密放入仓库。

“令牌”在这里指一种登录凭据，应像密码一样保密。

3. 以为保存文件已经上传

保存只改变电脑中的文件；提交记录本地版本；推送才把提交发送到 GitHub。

解决方法：记住三个动作：保存、提交、推送。完成后在 GitHub 页面核对。

4. 忘记先取得最新版本

如果另一台电脑或其他成员已经推送新内容，而你仍在旧版本上工作，之后可能出现冲突。

“冲突”表示 Git 无法自动判断两组修改应该怎样组合。

解决方法：开始工作前先取得远程最新版本，再修改文件。

5. 提交说明看不懂

如果每次只写“修改”“更新”，以后无法知道具体做了什么。

解决方法：说明对象和目的，例如“发布第五章 GitHub 入门”。

6. 不知道网页上的按钮作用

GitHub 页面包含 Code、Issues、Pull requests、Actions 等入口，新手不需要一次学完。

解决方法：先认识仓库首页、文件列表和提交历史。需要某项功能时再学习，不要同时打开所有功能。

7. 登录或推送失败

可能原因包括账号未登录、权限不足、网络问题或身份验证过期。

“身份验证”是系统确认你是谁的过程。

解决方法：保留完整错误提示，确认账号、仓库地址和权限，不要反复输入或公开密码。

8. 害怕操作后无法恢复

新手常担心点错按钮会毁掉项目。

解决方法：先在练习仓库操作；重要操作前检查仓库名称和分支；不确定时停止并询问。Git 的版本历史可以帮助恢复，但仍要谨慎处理删除和覆盖。

七、本章练习

练习 1：区分 Git 与 GitHub

分别用一句话解释 Git 和 GitHub。
完成标准：回答中包含“本地记录”和“在线保存与协作”，没有把两者说成同一个工具。

练习 2：判断哪些内容不能公开

从下面内容中找出不应该放入公开仓库的项目：

- 公开学习笔记。
- 登录密码。
- 示例 Markdown 文档。
- 访问令牌。
- 已确认可以公开的流程图。

完成标准：能指出密码和访问令牌不能公开。

练习 3：描述本项目流程

不用照抄正文，写出 GPT-Codex-Guide 从完成一章到 GitHub 验收的步骤。
完成标准：包含检查、Git 提交、推送和网页验收。

八、本章总结

【本章总结】
GitHub 是在线保存项目版本、展示内容并支持协作的平台。
它与 Git 的区别是：

- Git 主要在本地记录文件变化。
- GitHub 主要在线保存 Git 仓库并支持分享和协作。

GitHub 可以帮助项目保存在线副本、展示内容、保留历史、记录问题、支持多人工作，并让不同电脑继续同一个项目。
在 GPT-Codex-Guide 中，每完成一章，都会检查文件、创建一个提交、推送到 GitHub，再由用户人工验收。这让开发流程可以被追踪、重复和验证。

下一章预告

【下一章预告】
《第六章：Git 是什么》